Module 4

Problem Statement: Real vs. Fake News Classification

In the digital era, vast amounts of information circulate daily through online news platforms and social media. However, not all news items are reliable — many are misleading or fabricated ("fake news").

This poses significant challenges for readers, journalists, and organisations that seek to maintain credibility and trust.

Objective

The goal of this project is to **develop a machine learning model** that can automatically classify a news article as **real (authentic)** or **fake (misleading)** based on its textual content.

Dataset Description

Each record in the dataset contains:

Column	Description
`title`	Headline of the news article
`text`	Body/content of the news article
`label`	Class label: `0` = Real, `1` = Fake

Text → Clean → Tokenize → Encode → Model → Train → Evaluate → Predict

In text classification, cleaning means preparing raw text so that it can be effectively understood by the model.

Input:

```
"The Movie Was Fantastic!!! 100% recommended :) #BestMovie"
```

After cleaning:

```
"movie fantastic recommended"
```

- **Lowercasing**
- **Remove Punctuation, Numbers, and Special Characters**

Sentence	Meaning for classification
“The movie was fantastic”	positive sentiment
“movie fantastic”	still clearly positive

The **sentiment** (positive) doesn’t change even after removing *“the”* and *“was”*.

Tokenization means breaking down a text (sentence, paragraph, or document) into **smaller units called tokens**.

A **token** can be:

- a **word**
- a **subword** (for BERT-type models)
- or even a **character**

These tokens are what your model “sees” instead of raw text.

Because **models can only handle numbers**, not text.

After tokenization, each token is converted to a **token ID** (integer) from a vocabulary:

```bash
{'the':1, 'movie':2, 'was':3, 'fantastic':4, 'and':5, 'emotional':6}
Sentence: "the movie was fantastic" → [1, 2, 3, 4]
```

What are Word Embeddings?

- Represent words as dense vectors instead of one-hot encodings.
- Capture semantic and syntactic relationships.
- Similar words are close in the embedding space.

Motivation

- One-hot vectors are sparse and don’t reflect meaning.
- Embeddings learn meaning from co-occurrence patterns in text.
- Example: king - man + woman ≈ queen.

Word2Vec (2013)

- Two models: CBOW and Skip-Gram.
- CBOW: Predict target from context words.
- Skip-Gram: Predict context from target word.
- Training Objective: Maximize probability of correct word-context pairs.

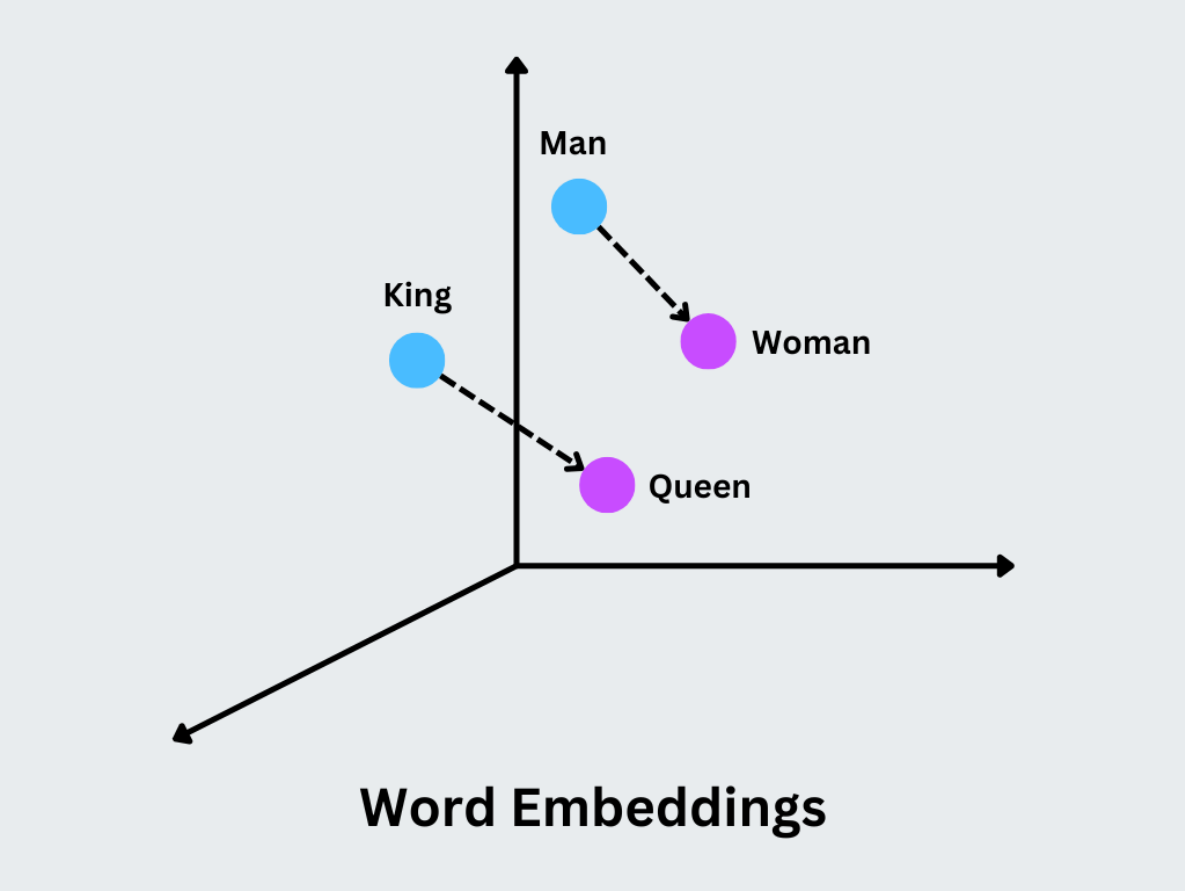
GloVe (2014)

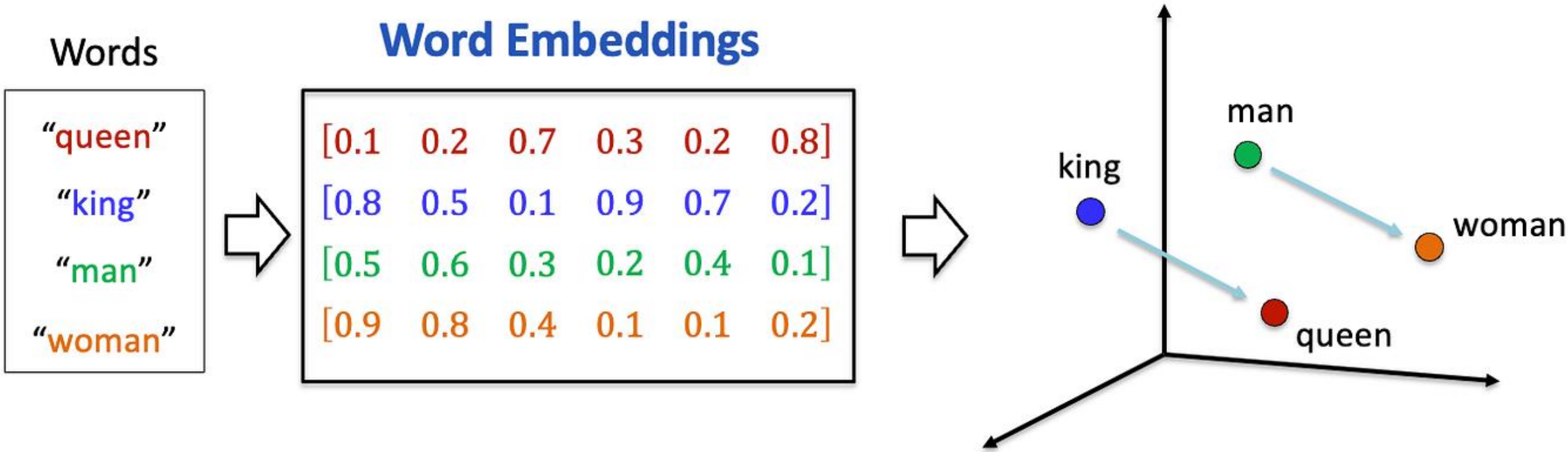
- Combines local context with global co-occurrence statistics.
- Minimizes difference between predicted and actual co-occurrence counts.
- Captures semantic and analogical relationships.

Contextual Example

- Word: bank
- He sat by the river bank → different meaning from
- She deposited money in the bank.

A word embedding is a way to represent words as dense numerical vectors that capture their meaning and relationship with other words.




Words
"queen"
"king"
"man"
"woman"
Word Embeddings
[0.1 0.2 0.7 0.3 0.2 0.8]
[0.8 0.5 0.1 0.9 0.7 0.2]
[0.5 0.6 0.3 0.2 0.4 0.1]
[0.9 0.8 0.4 0.1 0.1 0.2]
man
king
woman
queen

Real news:

> "Government announces new policy for rural development."

Fake news:

> "Government secretly bans all rural schools next month."

Step 1. Tokenization

Break each text into words (tokens):

Real News Tokens	Fake News Tokens
['government', 'announces', 'new', 'policy', 'for', 'rural', 'development']	['government', 'secretly', 'bans', 'all', 'rural', 'schools', 'next', 'month']

Step 2. Word Embedding Representation

Now each word is converted into a **vector of numbers** that shows meaning.

Let's show a *tiny imaginary embedding space (3D)* for a few words:

Word	Embedding (3 numbers)
government	[0.90, 0.20, 0.30]
announces	[0.70, 0.80, 0.10]
policy	[0.85, 0.75, 0.20]
secretly	[0.15, 0.90, 0.80]
bans	[0.10, 0.85, 0.75]
schools	[0.50, 0.60, 0.55]

Concept	Example
Raw text	"Government secretly bans all rural schools next month."
Tokens	['government','secretly','bans','rural','schools']
Embeddings	Each word → numeric vector capturing meaning

Term	Meaning (Simple Explanation)
Token	A word or subword after tokenization.
Vocabulary (Vocab)	The set of all unique tokens in your dataset.
Token ID / Index	Each token is given a unique integer ID.
Embedding	A numeric vector that represents a word's meaning.
Embedding Dimension	The size (length) of each word vector.

- **A word embedding turns each token from a vocabulary into a vector of size embedding_dim.**
- **These vectors lie in a semantic space where similar words are close together.**

Let's take **3 short sentences** (for a fake news classifier):

1. "Government announces new policy"
2. "Fake news spreads fast"
3. "Government denies the rumor"

We'll assume:

- Batch size = 3 (3 sentences)
- Vocabulary size = 10 (only 10 unique words)
- Maximum sentence length = 5
- Embedding dimension = 4

Convert each word into a **token ID** from the vocabulary.

Word	Token ID
government	1
announces	2
new	3
policy	4
fake	5
news	6
spreads	7
fast	8
denies	9
rumor	10

```
X = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [1, 9, 10]
]
```

🔹 Step 3: Padding

All sequences must have the same length (max_len = 5).

We pad with zeros (0 = `<PAD>` token):

```lua

[[1, 2, 3, 4, 0],
 [5, 6, 7, 8, 0],
 [1, 9, 10, 0, 0]]
```

➡️ **Shape:** (3, 5)

→ 3 sentences × 5 tokens each

🔹 Step 4: Embedding Layer

Now we pass this to an **Embedding layer**:

```
Embedding(input_dim=10, output_dim=4, input_length=5)
```

- `input_dim = 10` → vocab size (10 words + padding)
- `output_dim = 4` → each word represented as 4 numbers
- `input_length = 5` → each sentence = 5 tokens

The layer looks up each token ID in an **embedding matrix** of shape:

```scss
(10, 4)
```

So each word ID → 4D vector.

🔹 Step 5: Embedding Output

For each input sentence (5 words), you now get **5 embeddings** (each of size 4).

```
Input shape  : (3, 5)
Output shape : (3, 5, 4)
```

Example output:

```python
[
 [[0.10, 0.25, 0.70, 0.55],   # government
  [0.60, 0.40, 0.10, 0.22],   # announces
  [0.75, 0.90, 0.33, 0.11],   # new
  [0.44, 0.20, 0.60, 0.12],   # policy
  [0.00, 0.00, 0.00, 0.00]],  # pad

 [[0.91, 0.20, 0.13, 0.70],
  [0.60, 0.10, 0.55, 0.44],
  [0.80, 0.32, 0.15, 0.40],
  [0.72, 0.10, 0.11, 0.10],
  [0.00, 0.00, 0.00, 0.00]],

 ...
]
```

Step	Operation	Example Shape	Description
1	Raw Text	`(3,)`	3 sentences
2	Tokenization	`(3, variable)`	each word → ID
3	Padding	`(3, 5)`	all sentences = same length
4	Embedding Lookup	`(3, 5, 4)`	each word → 4-D vector

Text = a sequence of dependent meanings.

```
Sentence A: "The movie was not good."
Sentence B: "The movie was good."
```

```
Word 1 → Word 2 → Word 3 → Word 4
   ↓        ↓        ↓        ↓
 [Hidden state passes through each step]
```

Sentence A:

> "The company increased its profit."

Sentence B:

> "The company hardly increased its profit."

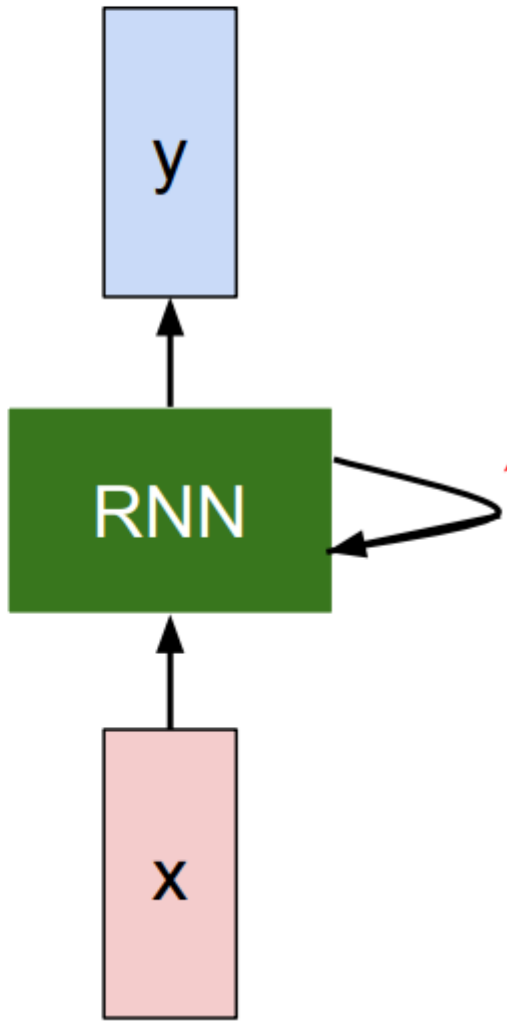
Both contain the same words: *company, increased, profit*

But "**hardly**" changes the meaning — only a **sequential model** captures that.

Sequence models

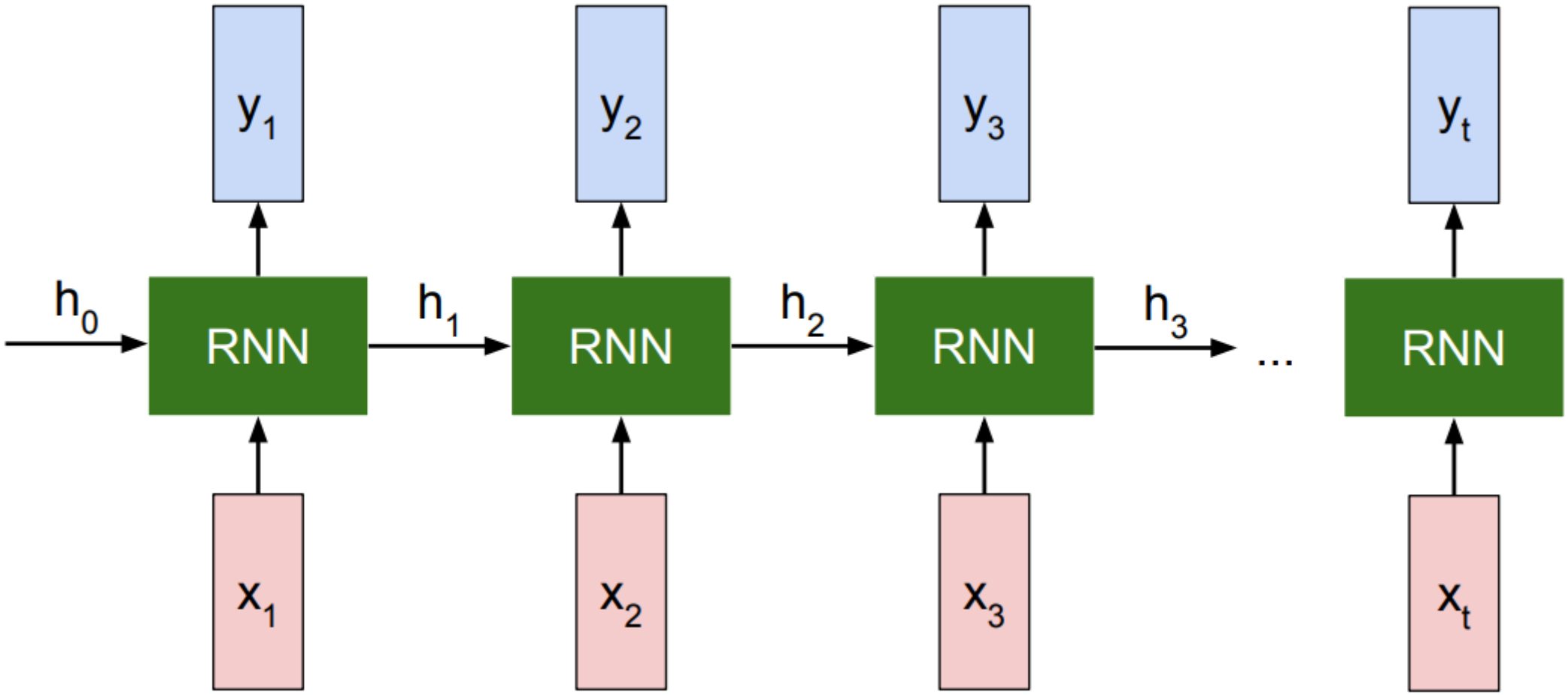
Model	Think of it as…
RNN	Reading one word at a time, remembering the past like a short memory chain.
Transformer	Reading the whole sentence at once and deciding which words are important using attention.

Recurrent Neural Network



Key idea: RNNs have an "internal state" that is updated as a sequence is processed

Unrolled RNN



RNN hidden state update

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

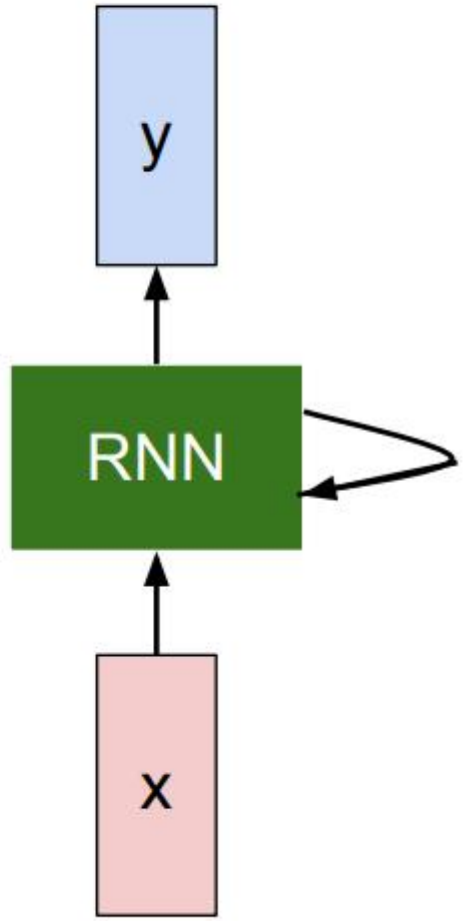
$$h_t = f_W(h_{t-1}, x_t)$$

new state

some function with parameters W

old state

input vector at some time step



RNN output generation

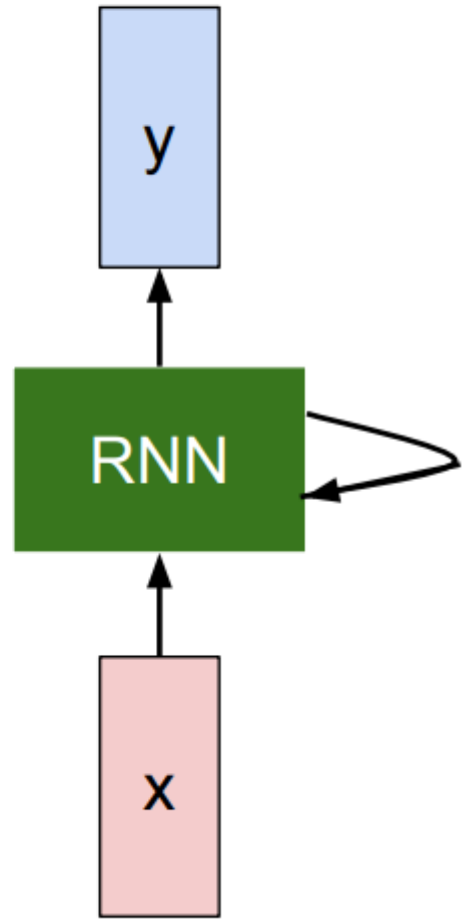
We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$y_t = f_{W_{hy}}(h_t)$$

output

new state

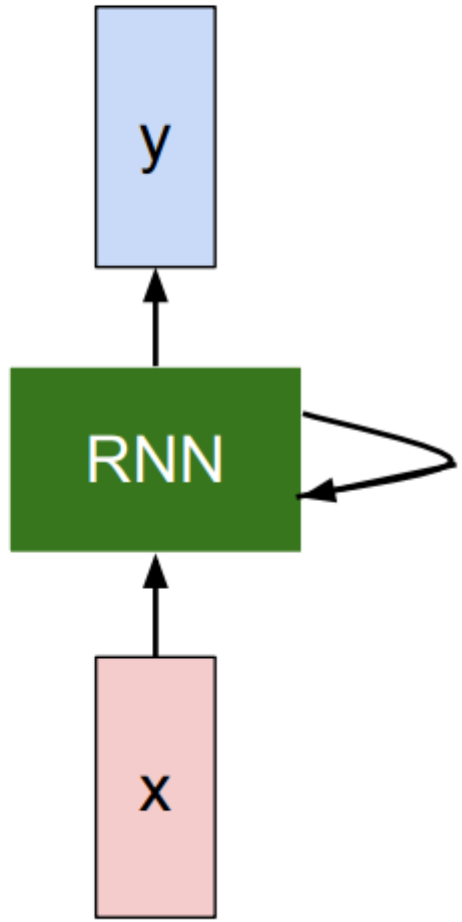
another function with parameters W_o



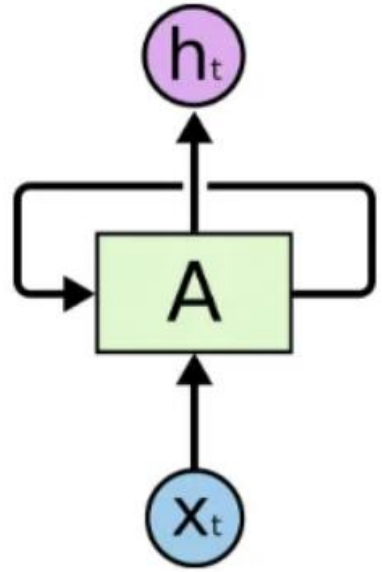
Recurrent Neural Network

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

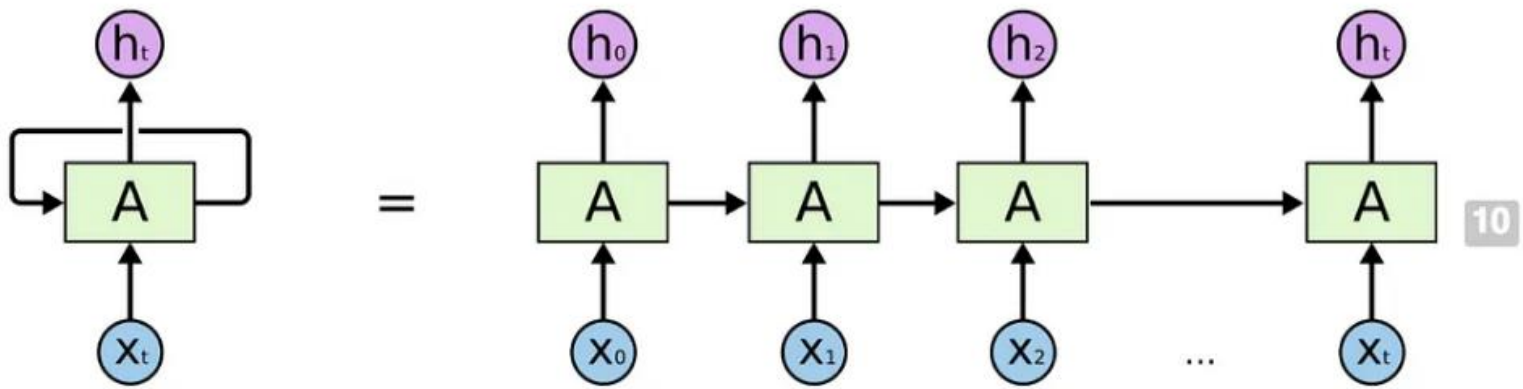


Notice: the same function and the same set of parameters are used at every time step.



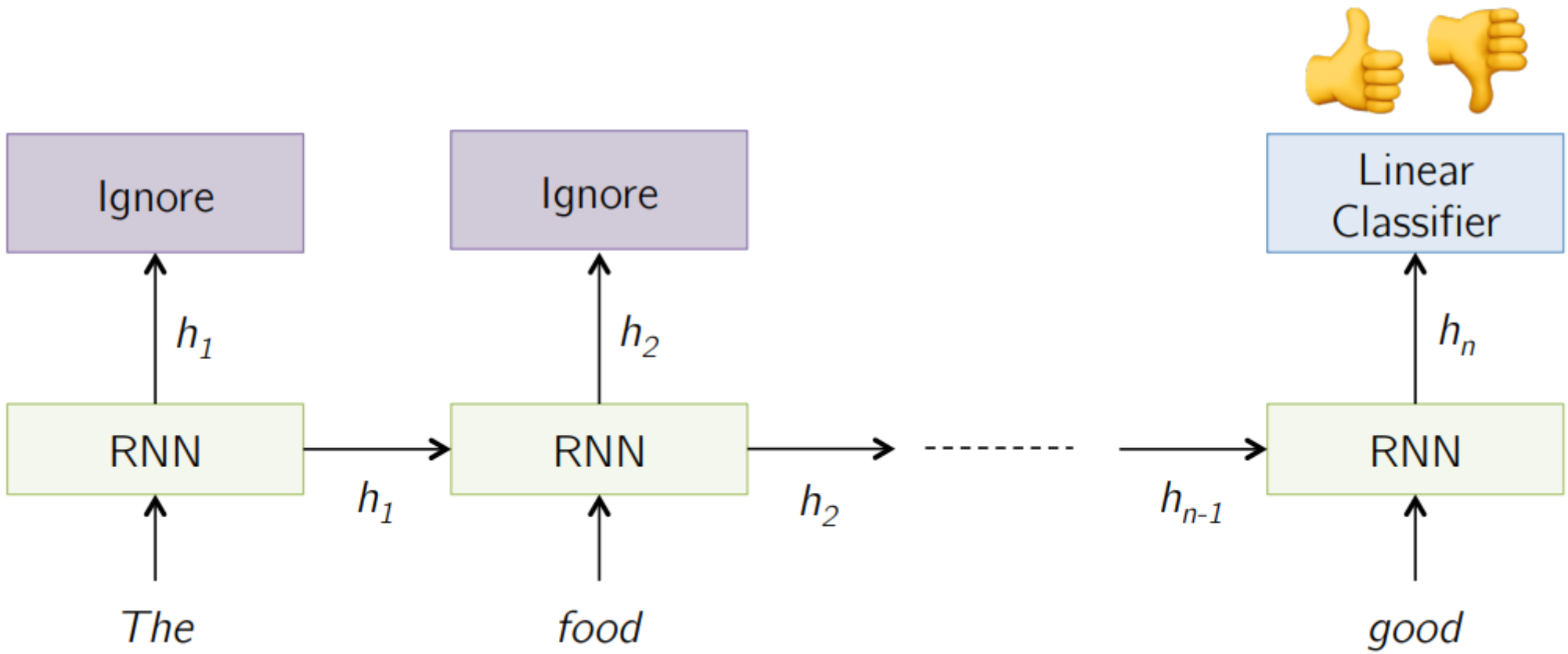
7

Recurrent Neural Networks have loops.



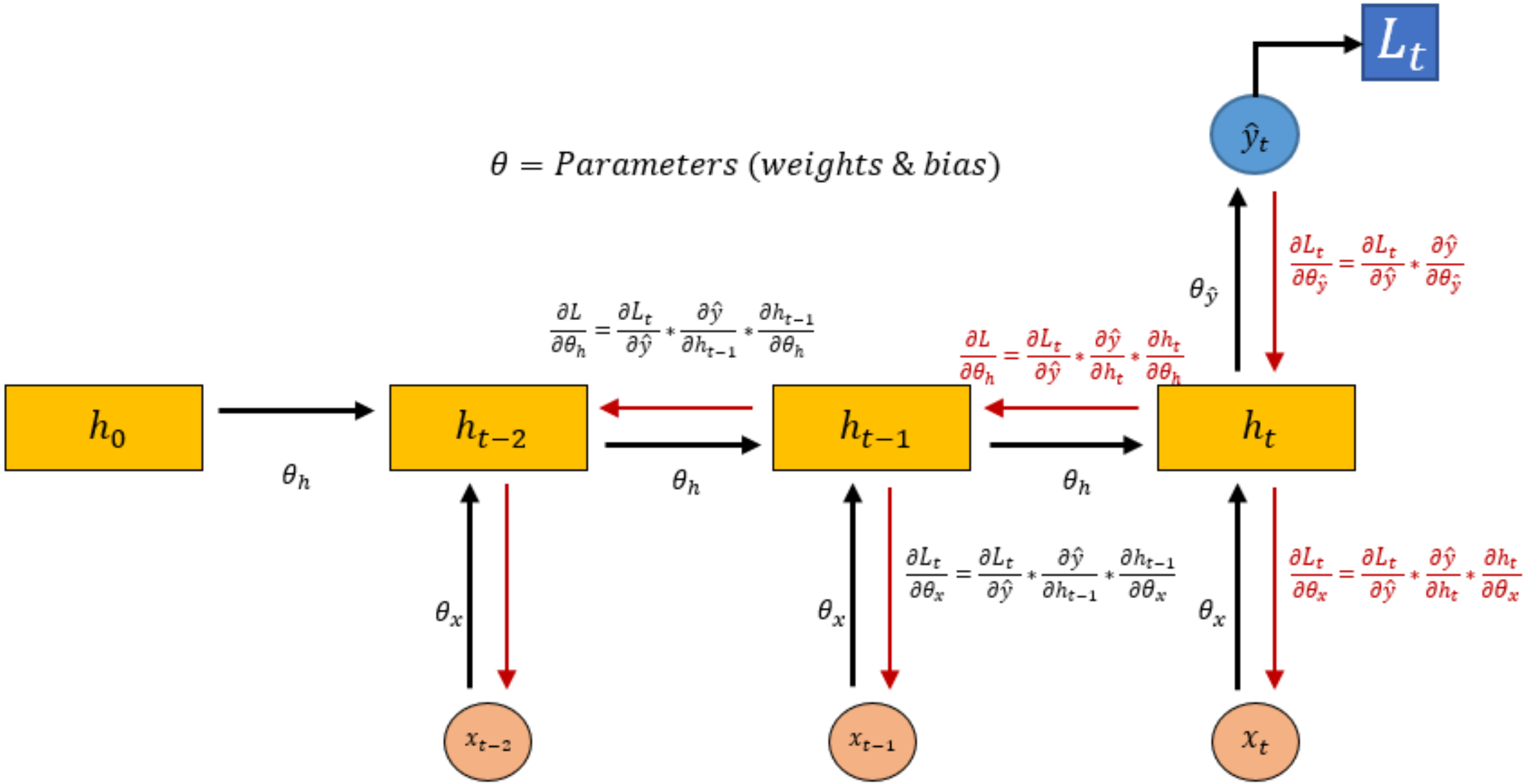
10

Sentiment Classification



Features of RNN

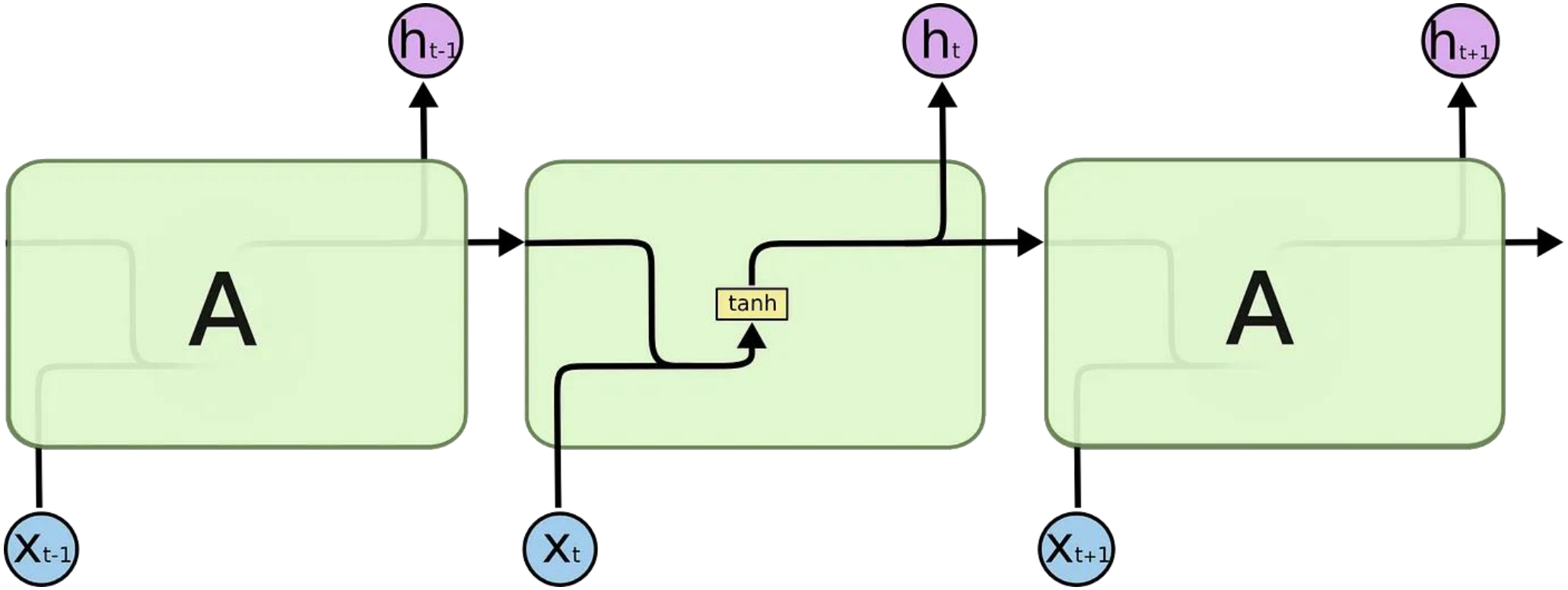
- The sequence nature of features is handled by computing a forward pass by propagating the state (hidden) across the time-dependent features.
- Forward pass → Unroll in time
- Backpropagation through time
- Different configurations depending upon the application
- Now, even though RNNs are quite powerful, they suffer from **Vanishing gradient problem** which hinders them from using long-term information, they are good for storing memory of 3-4 instances of past iterations but a larger number of instances don't provide good results so we don't just use regular RNNs. Instead, we use a better variation of RNNs: **Long Short Term Networks(LSTM).**

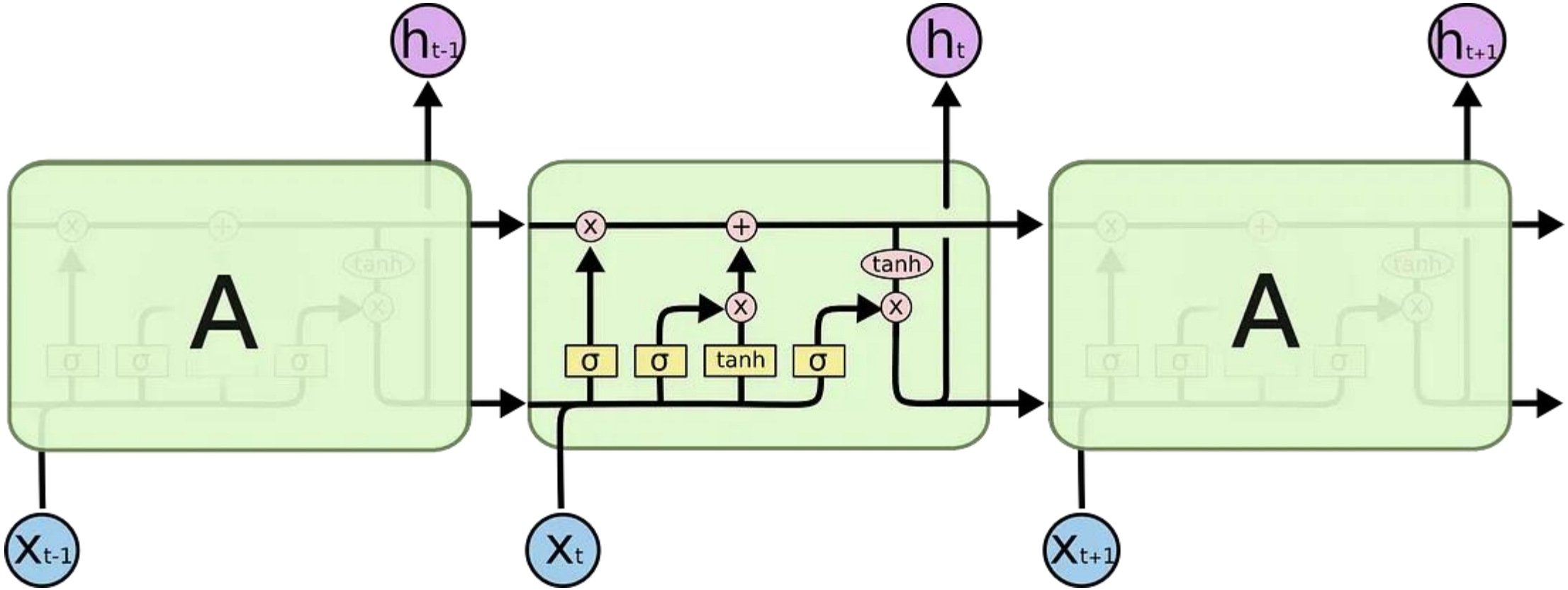

$\theta = Parameters\ (weights\ \&\ bias)$
L_t
$\hat{y}_t$
$\theta_{\hat{y}}$
$\frac{\partial L_t}{\partial \theta_{\hat{y}}} = \frac{\partial L_t}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial \theta_{\hat{y}}}$
h_0
h_{t-2}
h_{t-1}
h_t
θ_h
θ_h
θ_h
$\frac{\partial L}{\partial \theta_h} = \frac{\partial L_t}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial h_{t-1}} * \frac{\partial h_{t-1}}{\partial \theta_h}$
$\frac{\partial L}{\partial \theta_h} = \frac{\partial L_t}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial h_t} * \frac{\partial h_t}{\partial \theta_h}$
θ_x
θ_x
θ_x
$\frac{\partial L_t}{\partial \theta_x} = \frac{\partial L_t}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial h_{t-1}} * \frac{\partial h_{t-1}}{\partial \theta_x}$
$\frac{\partial L_t}{\partial \theta_x} = \frac{\partial L_t}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial h_t} * \frac{\partial h_t}{\partial \theta_x}$
x_{t-2}
x_{t-1}
x_t

The main limitation of RNNs is that RNNs can’t remember very long sequences and get into the problem of vanishing gradient.

- What is the vanishing gradient problem?

Simple RNN





Neural Network Layer

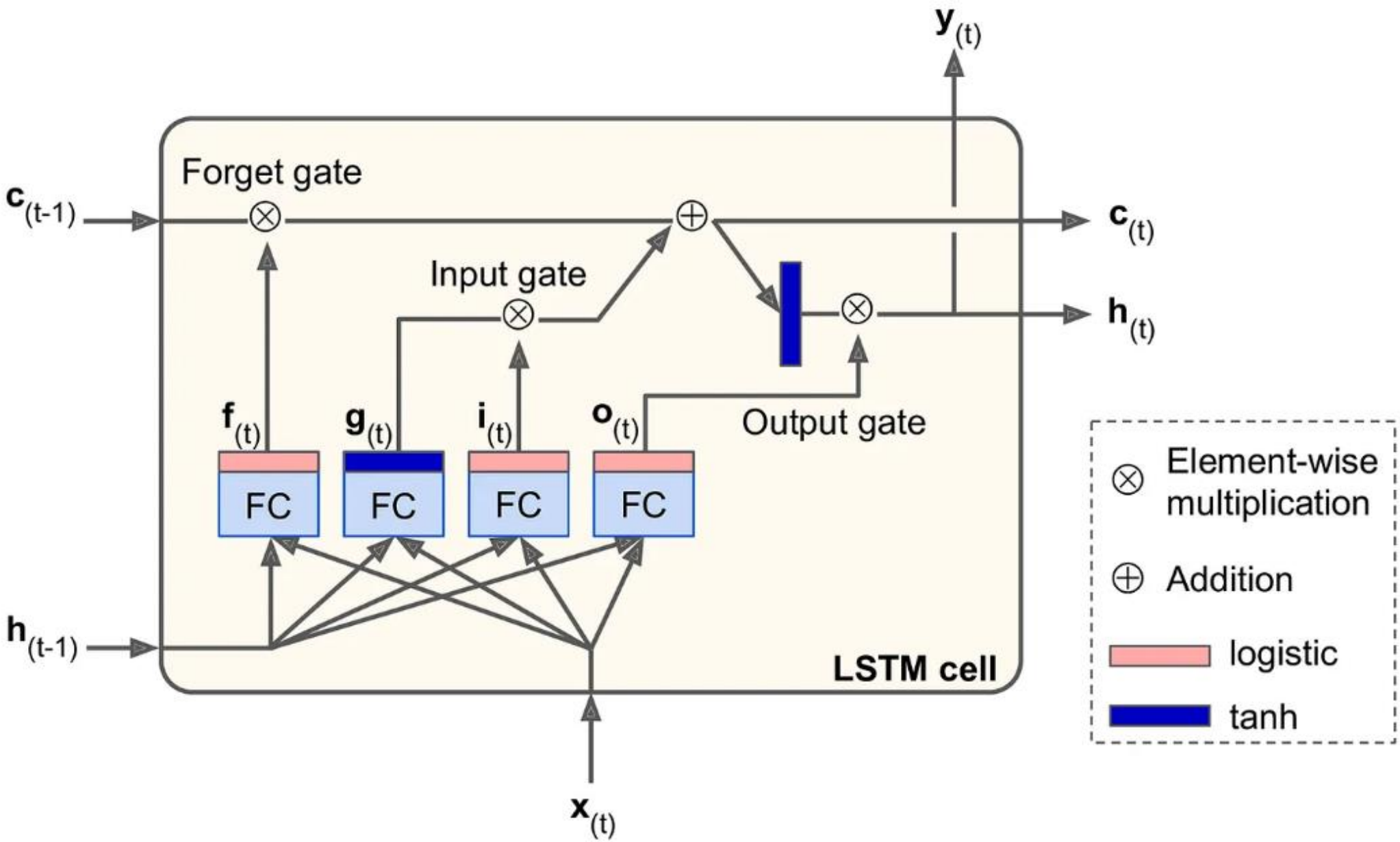
Pointwise Operation

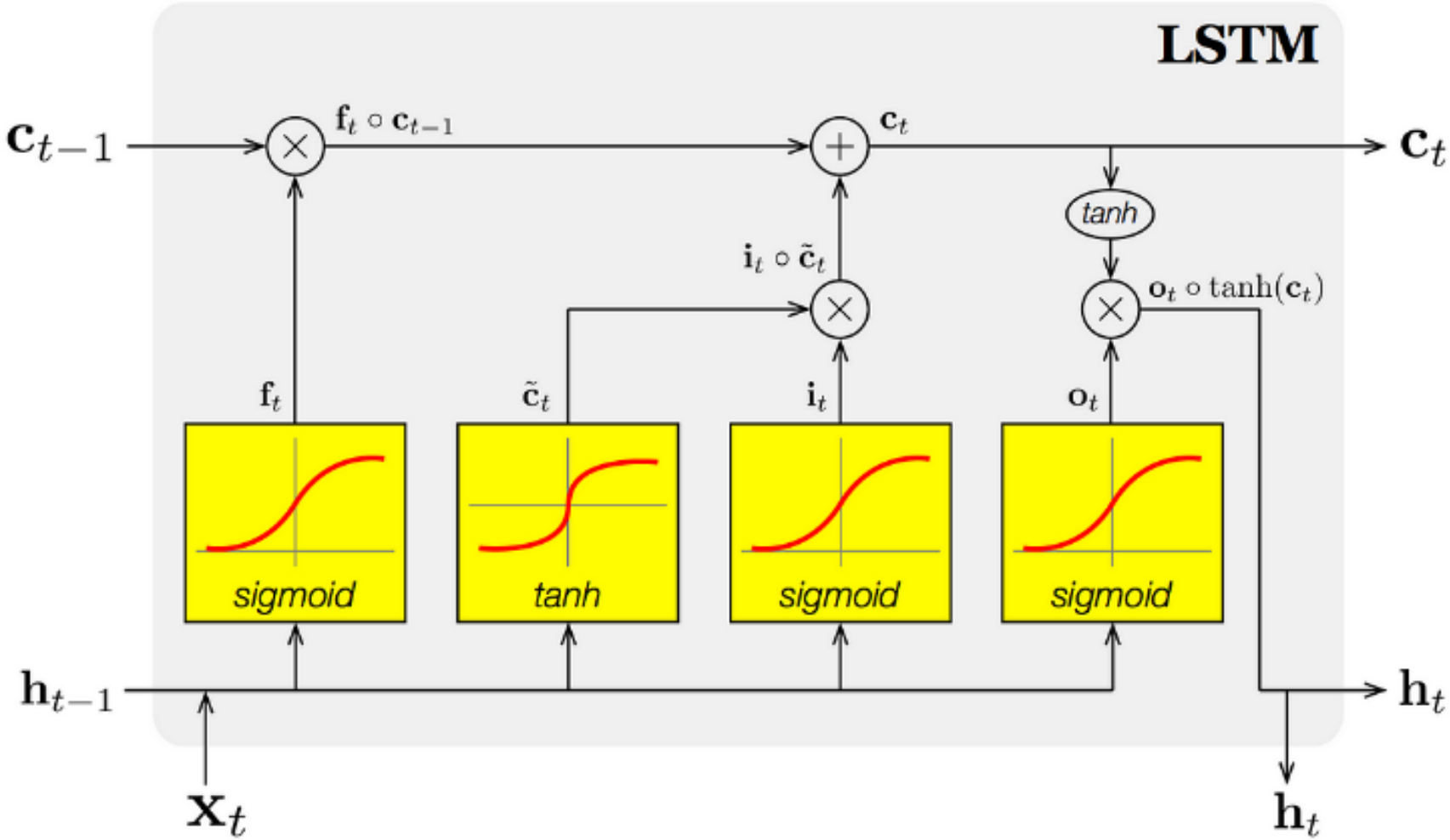
Vector Transfer

Concatenate

Copy

LSTM Architecture





Gating variables

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_t\right)$$
$$\mathbf{i}_t = \sigma\left(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\right)$$
$$\mathbf{o}_t = \sigma\left(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\right)$$

Candidate (memory) cell state

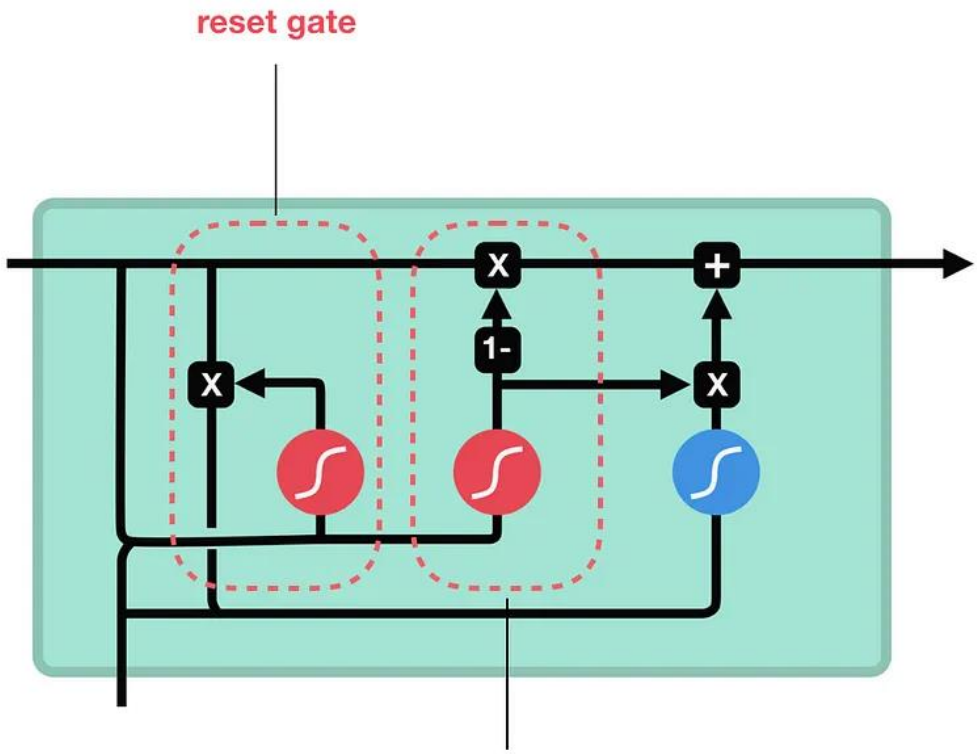
$$\tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c\right)$$

Cell & Hidden state

$$\mathbf{c}_t = \mathbf{f}_t \circ \mathbf{c}_{t-1} + \mathbf{i}_t \circ \tilde{\mathbf{c}}_t$$
$$\mathbf{h}_t = \mathbf{o}_t \circ \tanh(\mathbf{c}_t)$$

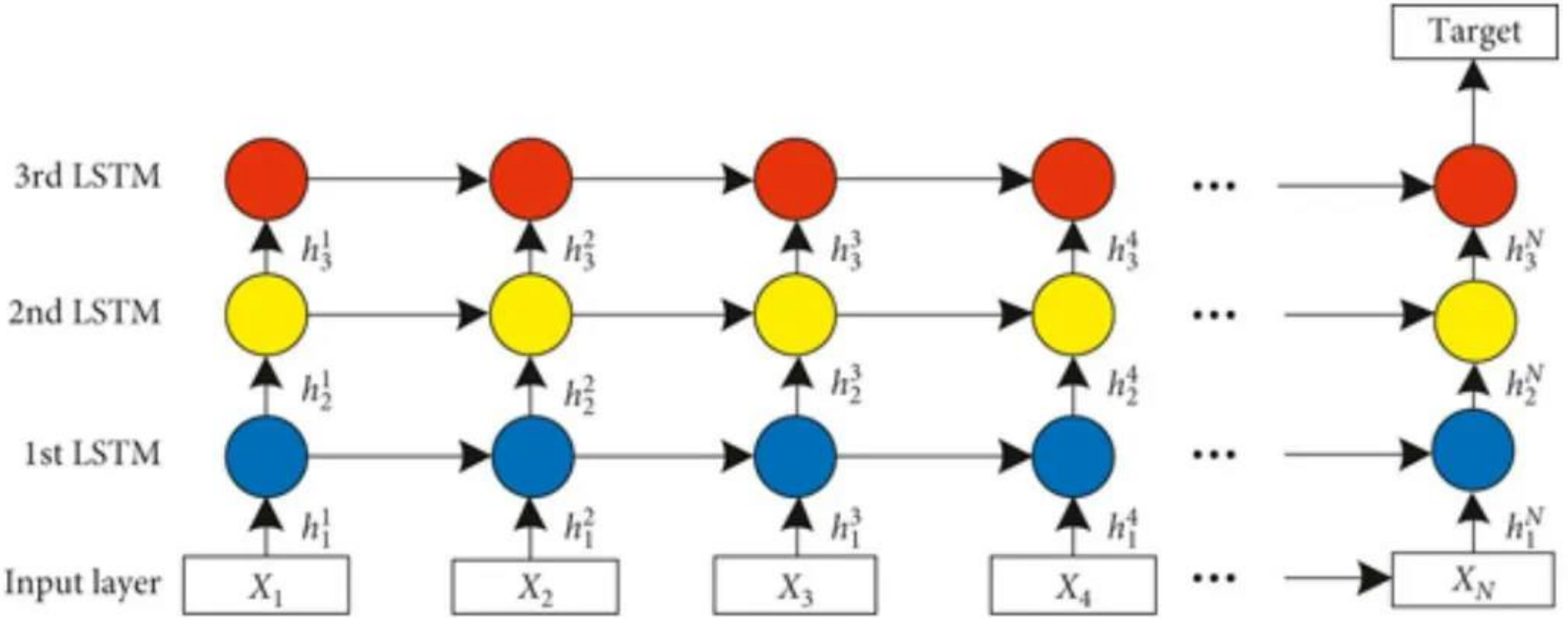
LSTM parameter number = 4 × ((x **+** h**) ×** h **+** h**)**

Gated Recurrent Units (GRU)



- **Architecture**: GRUs are less complex than LSTMs and have fewer gates and parameters. GRUs combine the cell and hidden states into one entity, while LSTMs maintain separate cell and hidden states.
- **Speed**: GRUs are faster to compute than LSTMs.
- **Memory**: GRUs use less memory than LSTMs.
- **Training**: GRUs are generally easier and faster to train than LSTMs.
- **Accuracy**: LSTMs are more accurate when using datasets with longer sequences.

Stacked RNNs



Concluding remarks

- Text classification automatically assigns labels such as sentiment or topic to textual data.
- It involves preprocessing text, extracting features, training a model, and evaluating its performance.
- Classical models like Naive Bayes and SVM work for small datasets, while deep learning and transformer models capture complex context.
- Model performance is measured using accuracy, precision, recall, F1-score, and confusion matrix.
- The main challenges include imbalanced data and multilingual text handling.